Software Impacts 30 (2026) 100863

Contents lists available at ScienceDirect

Software Impacts

journal homepage: www.journals.elsevier.com/software-impacts

Original software publication

ResQConnect: AI-assisted disaster response and task synthesis system with agentic RAG

Chemini Mallikarachchi, Savinu Aththanayake, Janeesha Wickramasinghe, Sajeev Kugarajah, Dulani Meedeniya *

University of Moratuwa, Sri Lanka

ARTICLE INFO

Keywords:
Agentic RAG
Disaster response systems
Edge AI
Human-in-the-loop systems
Humanitarian logistics
Reinforcement learning

ABSTRACT

Disaster response operations demand rapid, coordinated decision-making under degraded infrastructure and limited connectivity. Existing systems rely on manual workflows and fragmented information, leaving coordinators ill-equipped to process requests efficiently. ResQConnect is a human-centred AI-assisted decision-support platform integrating four components: an Agentic RAG workflow that converts unstructured help requests into field-executable task plans; an AET multi-commodity routing engine for dynamic resource allocation; a fine-tuned lightweight language model enabling offline guidance on mobile devices; and an RL-based retrieval policy that minimizes unnecessary cost. Human coordinators retain full decision authority throughout. The system contributes to SDG 11 by enhancing resilience in climate-related emergencies.

Code metadata

Current code version	v1.0
Permanent link to code/repository used for this code version	https://github.com/SoftwareImpacts/SIMPAC-2026-76
Permanent link to Reproducible Capsule	
Legal Code License	MIT License https://github.com/MinionZ-UOM/ResQConnect-final/blob/main/LICENSE
Code versioning system used	git
Software code languages, tools, and services used	Python, JavaScript (Next.js), FastAPI, LangGraph, Celery, Redis, ChromaDB, Firebase, TensorFlow Lite, MediaPipe, Hugging Face Transformers
Compilation requirements, operating environments & dependencies	Python ≥3.10, Node.js ≥18; Android device with Snapdragon 8 Gen 2 or equivalent for on-device inference
If available Link to developer documentation/manual	https://savinimz.github.io/ResQConnect-docs/
Support email for questions	nimetha.21@cse.mrt.ac.lk

1. Introduction

Flood and landslide disasters unfold rapidly, leaving emergency teams with little time and even fewer resources to coordinate an effective response. Help requests pour in through multiple channels in unstructured, often panicked forms. Infrastructure degrades, connectivity drops, and situational conditions shift faster than manual workflows can keep up with. Despite these well-known challenges, many existing disaster management systems still rely on static planning and human-driven coordination, making it difficult to process incoming requests at scale, prioritize urgent needs, or allocate resources where they are needed most.

Recent progress in large language models, retrieval-augmented generation, and edge AI offers a genuine path forward. These technologies make it possible to interpret unstructured inputs, retrieve grounded operational guidance, and support faster decision-making without replacing the human judgment that safety-critical environments demand. However, most existing solutions address only isolated aspects of the problem such as damage prediction, social media classification, or resource scheduling, and none deliver an integrated, end-to-end workflow capable of operating reliably under real disaster conditions. A further overlooked challenge is operational cost: AI-assisted systems

* Corresponding author.
E-mail address: dulanim@cse.mrt.ac.lk (D. Meedeniya).

https://doi.org/10.1016/j.simpa.2026.100863
Received 24 April 2026; Accepted 3 September 2026
2665-9638/© 2026 The Author(s). Published by Elsevier B.V. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

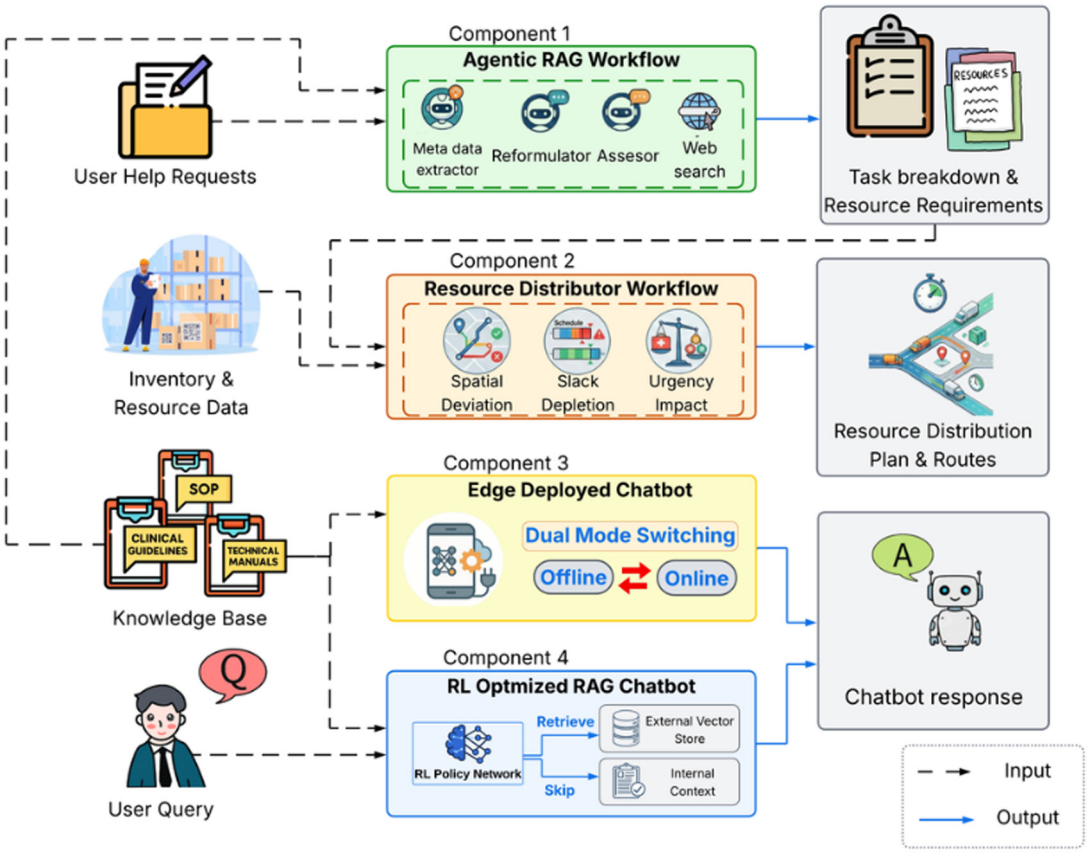


Fig. 1. High-level architecture diagram of the ResQConnect ecosystem.

that invoke retrieval indiscriminately accumulate unnecessary computation, which matters greatly in resource-constrained deployment environments [1].

ResQConnect [2] is built to close these gaps. Rather than treating each challenge in isolation, it coordinates four purpose-built components into a single pipeline that handles everything from interpreting a panicked text message to routing a relief vehicle across a degraded road network, while an RL-based retrieval policy continuously learns when fetching new information is worth the cost and when existing context is already sufficient. The system is grounded in domain knowledge sourced from disaster management authorities and evaluated against realistic flood and landslide scenarios from the 2025 Sri Lanka disaster response [3].

In this paper, Section 2 describes the system architecture and core components, Section 3 discusses software contributions and impacts, and Section 4 outlines limitations and future work.

2. Software description

2.1. System architecture

ResQConnect integrates four subsystems into an end-to-end pipeline as depicted in Fig. 1. Citizen help requests enter through a web or mobile interface. The Agentic RAG Workflow interprets each request and produces a structured task breakdown with associated resource requirements [4]. The Resource Distributor Workflow translates these requirements into optimised multi-depot dispatch plans. In parallel, the Edge-Deployed Chatbot provides offline guidance on mobile devices when cloud connectivity is degraded or unavailable. The reinforcement component operates as a decision-control layer within the chatbot. Its primary role is to determine whether external knowledge retrieval is required for a given user request. All system outputs are presented as recommendations to authorised human operators, who retain final decision authority.

2.2. Agentic RAG workflow for task synthesis

When a citizen sends in a help request, it rarely arrives in a clean or well-structured form. People under stress write short, emotional, and often incomplete messages. The workflow is designed to handle this kind of input.

The Agentic RAG workflow pipeline runs through five nodes as shown in Fig. 2. First, a Meta Node reads the incoming message and picks out what matters most: the type of disaster, where it is happening, and how urgent the situation seems. From there, a Filtered Retriever searches through a knowledge base built from flood and landslide-specific Standard Operating Procedures, Sphere guidelines, and technical advisories. If what comes back is too vague or does not contain the kind of step-by-step procedural detail needed to act on, an Assessor Node catches this and routes the query to a Reformulator Node. The Reformulator rewrites the original message into a cleaner, more focused query and tries the retrieval again. Once the retrieved context passes the adequacy check, a Task Generator produces a concrete breakdown of actions for the relevant response agencies along with the resource requirements attached to each task [5].

The outputs follow a fixed metadata structure covering incident type, location, urgency, and resource needs. This makes it straightforward for the routing module to pick up where the RAG workflow leaves off. The knowledge base itself was put together with input from officers at Sri Lanka's National Disaster Relief Services Centre, so the tasks the system generates reflect closely how response actually works on the ground rather than what a general-purpose model might assume.

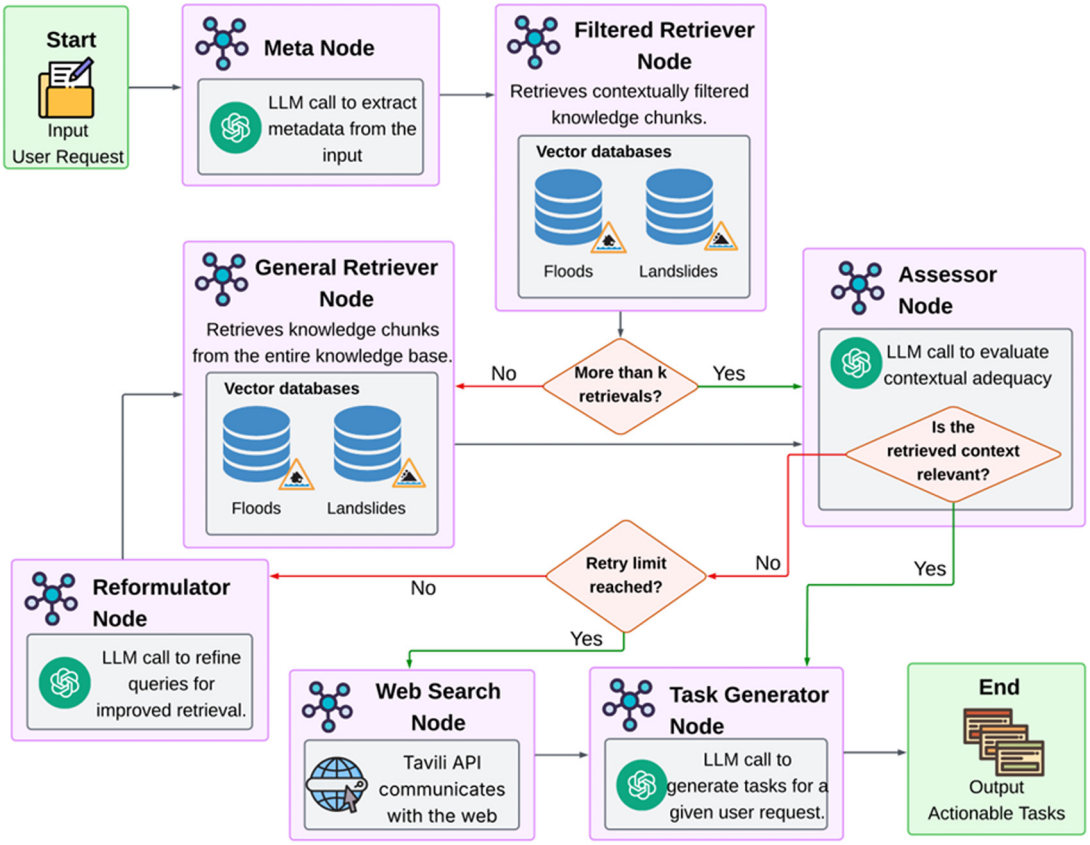


Fig. 2. Communication flow of the multi-agent RAG pipeline.

2.3. Adaptive event-triggered multi-commodity routing

The resource-distribution component of ResQConnect is designed for dynamic disaster environments where requests arrive continuously, conditions change rapidly, and resources must be allocated under strict constraints. Each affected location may require multiple resource types (e.g., food, water, medicine), and the problem is modelled as a multi-depot, multi-commodity routing task. Vehicles depart from depots with limited capacity and must deliver complete resource bundles to each location. The system assumes atomic fulfilment, meaning each request is served by a single vehicle, as partial deliveries are often impractical in disaster scenarios. A key challenge is determining when to re-optimise routes. While frequent re-optimisation improves responsiveness, it is computationally expensive and leads to unstable routes that are difficult for field teams to follow. Conversely, fixed or greedy update strategies reduce computational cost but may fail to respond to urgent changes. To address this, ResQConnect introduces an Adaptive Event-Triggered (AET) routing mechanism that balances responsiveness, efficiency, and stability.

The AET policy computes a disruption score based on three factors: request urgency, spatial deviation from existing routes, and remaining schedule slack. The disruption score can be expressed as a weighted combination of these factors:

$$D(t) = w_1 \cdot \Phi_{\text{urgency}} + w_2 \cdot \Phi_{\text{spatial}} + w_3 \cdot \Phi_{\text{slack}}$$

where w_1, w_2, w_3 are tunable weights that control the relative importance of urgency, spatial deviation, and system slack. This score is compared against a decaying adaptive threshold. The adaptive threshold is defined as an exponentially decaying function:

$$\Theta(t) = \Theta_0 \cdot e^{-\alpha(t - t_{\text{last}})}$$

where Θ_0 is the initial threshold immediately after the last re-optimis ation, α is the decay rate, and t_{last} is the time of the most recent global optimisation. This decay mechanism gradually increases the system's sensitivity to new events over time, ensuring that re-optimisation is triggered when necessary without causing excessive instability. If the threshold is exceeded, global re-optimisation is triggered; otherwise, the request is handled via local route insertion. This allows urgent or high-impact events to trigger replanning, while minor changes are absorbed efficiently without unnecessary solver calls. The optimisation objective incorporates multiple practical considerations, including travel time, priority-weighted service delays, unmet demand, and route stability. High-urgency requests are given greater importance, while deviations from existing routes are penalised to maintain operational consistency. This ensures a balance between efficiency and realistic field execution.

Within the overall system, the routing module operates downstream of the Agentic RAG component. Structured requests are passed along with real-time data such as depot inventories, vehicle states, and travel conditions. The module outputs dispatch plans specifying vehicle assignments, delivery sequences, and estimated arrival times, adapting continuously to evolving conditions.

2.4. Edge-deployed small language model

A compressed, domain-specialised small language model (SLM) enables offline guidance on mobile devices during connectivity outages, a frequent condition in disaster-affected areas. Qwen2.5-0.5B [6] was selected from a benchmarked pool of sub-1B models (Qwen2.5-0.5B, TinyLlama-1.1B, Phi-1.5, Gemma-3-1B-IT) based on its superior efficiency score, lowest latency per token, and best SQuAD F1 among candidates. The model was supervised-fine-tuned on a curated disaster Q&A dataset [7] of 580 high-quality instruction-answer pairs spanning 29 subcategories of flood and landslide response, then converted to the MediaPipe .task format with 4-bit quantization for on-device execution.

The mobile application implements dual-mode switching, transparently switching between cloud-backed RAG pipelines when connectivity is available and the on-device SLM when it is not.

2.5. RL-adaptive retrieval chatbot

The chatbot combines RAG with an RL-trained retrieval policy. In standard RAG systems, external document retrieval is invoked at every dialogue turn regardless of whether the system already holds the relevant information, incurring unnecessary latency and computational cost. The RL policy addresses this by deciding, at each turn, whether retrieval is genuinely needed given the current query and the knowledge already accumulated during the session.

The underlying architecture is a LangGraph pipeline in which each user turn passes through four sequential stages: a retrieval decision, an optional retrieval step, answer generation, and a summarisation step that updates a Grounded Knowledge Base (GKB).

The GKB is the system's running memory of verified facts for the current session. It stores only domain knowledge explicitly confirmed by retrieved source documents during the session, such as relevant evacuation procedures or safety guidelines, rather than raw conversation history. The summarisation step that maintains the GKB operates differently depending on what action was taken in the current turn. When retrieval occurs, the summariser may add new facts to the GKB. When retrieval is skipped, the summariser is prohibited from adding any new factual claims.

The retrieval decision at each turn is made by a lightweight neural policy. The policy observes the current session state, encoded into a vector representation. From this observation, the policy outputs a binary action: retrieve, meaning the system fetches relevant documents from the knowledge base before generating an answer, or skip, meaning the answer is generated directly from the existing GKB without any external lookup. The policy is trained in two stages. First, supervised pre-training on labelled conversation examples initialises the policy to a sensible baseline. Second, Proximal Policy Optimisation (PPO) fine-tunes the policy using a reward signal that jointly rewards answer quality and penalises unnecessary retrieval cost. Answer quality is scored by a judge language model on five dimensions: relevance to the query, factual correctness, groundedness in available evidence, completeness of the response, and handling of out-of-scope questions.

2.6. Implementation

The implementation of ResQConnect follows a modular, service-oriented architecture that integrates agentic AI workflows, optimisation modules, and edge-deployed models into a unified disaster response platform as shown in Fig. 3. The system is designed to operate in a hybrid cloud–edge environment, enabling both high-performance computation and low-latency local inference under constrained conditions.

At the application layer, the backend is implemented using FastAPI, which provides lightweight and high-throughput RESTful APIs for orchestrating interactions between system components. Agentic workflows, including the task synthesis pipeline and conversational chatbot, are implemented using LangGraph, allowing complex multi-step reasoning processes to be represented as structured execution graphs. The frontend is developed using Next.js and deployed on Vercel, enabling real-time interaction for operators and users. For data storage, Firebase is used, while ChromaDB serves as the vector database for storing embeddings of disaster-specific knowledge.

To support scalability and handle computationally intensive workflows, the system employs an asynchronous task execution architecture using Celery, with Redis acting as both the message broker and caching layer. Heavy agentic workflows, such as multi-step reasoning pipelines, retrieval operations, and routing computations, are offloaded to Celery workers. This design decouples request handling from long-running processes, ensuring that the API layer remains responsive

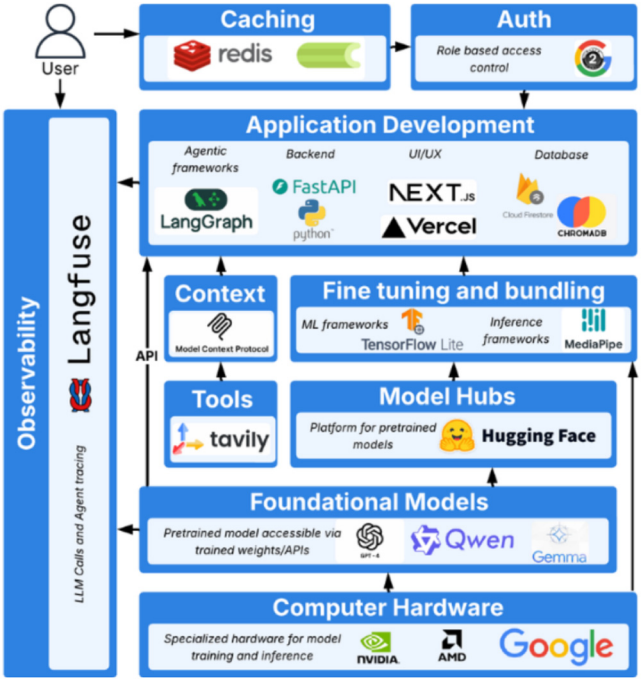


Fig. 3. Technology stack of the ResQConnect system.

even under high load. Redis further enhances performance by caching intermediate results, frequently accessed queries, and shared context across agents, reducing redundant computation and improving latency. To manage concurrency across the system, tasks are handled asynchronously through Celery workers, allowing multiple operations to run in parallel without blocking the main API. Redis is used to coordinate shared state, cache intermediate results, and control access to critical resources when needed.

A structured context management layer is used to standardise the exchange of metadata across components. Outputs from the Agentic RAG module, including incident type, location, urgency, and required resources, are encoded in a consistent schema and passed through APIs to downstream modules such as the routing engine. External tools and APIs are integrated through a unified interface, enabling the system to incorporate real-time data into decision-making.

The model integration layer supports both training and inference workflows. Lightweight frameworks such as TensorFlow Lite and MediaPipe are used to enable efficient deployment in edge environments. The edge-deployed small language model is optimised using 4-bit quantization and supports dual-mode, allowing the system to dynamically switch between local inference and cloud-based processing depending on resource availability and network conditions.

Pretrained models and embeddings are managed through a model hub layer, with Hugging Face used for accessing and integrating pre-trained models. The system supports multiple foundational models, enabling flexible trade-offs between accuracy and efficiency across different operational scenarios. At the infrastructure level, the system is deployed across both cloud and edge environments. Hardware acceleration using GPUs is leveraged for training and high-performance inference, while cloud services provide scalability and system reliability. This hybrid deployment approach ensures operational continuity even in disaster scenarios with limited connectivity. An observability layer is integrated to monitor system behaviour and performance. The system tracks agent execution flows, task queue processing, retrieval operations, and latency metrics, enabling debugging, performance optimisation, and continuous system improvement.

Table 1
Summary of key performance metrics across ResQConnect components.

Component	Metric	Value	Notes
AET Routing	Response time vs. continuous re-opt.	Within 7%–12%	Across 13 simulated scenarios
	Solver call reduction	Up to 85%	vs. always-reoptimise baseline
	Route nervousness reduction	50%–70%	Mid-route reassignments
Edge SLM (Qwen2.5-0.5B)	BLEU improvement	+236%	Fine-tuned vs. base model
	ROUGE-L improvement	+74%	Fine-tuned vs. base model
	F1 improvement	+83%	Fine-tuned vs. base model
	Semantic similarity improvement	+77%	Fine-tuned vs. base model
	On-device throughput	54.3 tokens/s	Snapdragon 8 Gen 2
	End-to-end latency	412 ms	Snapdragon 8 Gen 2
	Memory footprint	182 MB	4-bit quantization via MediaPipe
RL Retrieval Policy	Retrieval Rate	58.3%	RL Policy (vs. 100% always-retrieve)

3. Software impacts

ResQConnect addresses a clearly identified gap at the intersection of agentic AI, humanitarian logistics, edge computing, and reinforcement learning for disaster response. The software contributes to four communities.

Disaster management researchers and practitioners gain a system that improves coordination and decision-making in dynamic environments, enabling faster conversion of citizen requests into structured, actionable responses.

Humanitarian logistics researchers benefit from the AET routing module as a reference implementation of event-triggered re-optimisation for dynamic, multi-depot, multi-commodity vehicle routing. Across 13 simulated scenarios spanning low-to-extreme demand loads, AET remained within 7%–12% of the priority-weighted response time achieved by continuous re-optimisation while requiring up to 85% fewer solver calls and reducing system nervousness (mid-route reassignments) by 50%–70%. This makes the algorithm practically viable for real-time deployment on centralised infrastructure without specialised hardware.

Edge AI and mobile computing researchers gain a documented, reproducible pipeline for domain-specific SLM fine-tuning and on-device deployment targeting high-stakes, connectivity-constrained environments. The fine-tuned Qwen2.5-0.5B model achieved substantial improvements over the base model: BLEU +236%, ROUGE-L +74%, F1 +83%, and semantic similarity +77%. On-device deployment via MediaPipe on a Snapdragon 8 Gen 2 smartphone achieved 54.3 tokens/s throughput, 412 ms end-to-end latency, and a 182 MB memory footprint during inference, demonstrating the practical feasibility of emergency dialogue systems without cloud dependency.

Conversational AI and RL researchers gain a practical reference for applying reinforcement learning to adaptive retrieval in safety-critical, multi-turn dialogue systems. The trained policy reduces the retrieval rate to 58.3% of an always-retrieve baseline, demonstrating that retrieval cost can be cut substantially without degrading response usefulness (see Table 1).

Usability. A System Usability Scale (SUS) evaluation was conducted with 25 participants who interacted with a working prototype. Participants were recruited to represent a range of age groups, educational backgrounds, and provinces across Sri Lanka, with no prior familiarity with the system required. The mean SUS score was 75.2 out of 100, placing the system in the "Good" usability band. Specifically, 92% of participants found the system easy to use, 80% found its functions well integrated, and 80% felt confident using it as shown in Fig. 4. Only 8% considered it unnecessarily complex, and 4% reported encountering inconsistencies. Results indicate that the platform is accessible to users without specialised AI or software backgrounds, which is a necessary condition for real-world deployment.

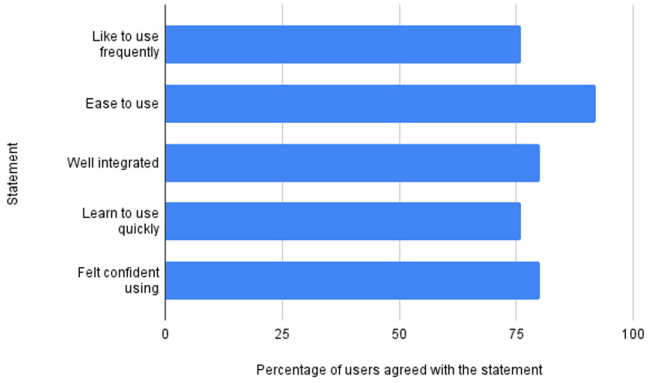


Fig. 4. Positive usability feedback from SUS evaluation.

4. Limitations and future work

ResQConnect has so far been tested only in simulated settings. Getting the system into an actual field deployment is the next step. Two limitations stand out as priorities. The routing module currently optimises for speed and urgency. It does not yet account for distributional fairness — the risk that optimising aggregate response time may systematically disadvantage geographically isolated communities. Incorporating a fairness constraint, such as bounding the Gini coefficient of priority-weighted response times across geographic sectors, would make this visible and correctable.

Extending the system to Sinhala and Tamil is equally important, and maintaining semantic fidelity across languages with very different grammatical structures will require careful validation beyond simple word-level translation. On the AI side, the RL retrieval policy currently depends on an LLM judge for reward signals, which is both costly and variable. Moving toward a lightweight reward model trained on human-labelled data would make the training process more stable and grounded.

CRediT authorship contribution statement

Chemini Mallikarachchi: Data curation, Methodology, Resources, Software, Validation, Visualization, Writing – original draft, Investigation. **Savinu Aththanayake:** Data curation, Investigation, Methodology, Resources, Software, Validation, -Visualization, Writing – original draft. **Janeesha Wickramasinghe:** Data curation, Investigation, Methodology, Resources, Software, Validation, Visualization, Writing – original draft. **Sajeev Kugarajah:** Data curation, Investigation, Methodology, Resources, Software, Validation, Visualization, Writing – original

draft. **Dulani Meedeniya:** Conceptualization, Funding acquisition, Investigation, Methodology, Resources, Supervision, Validation, Writing – review & editing, Project administration.

Declaration of Generative AI and AI-assisted technologies in the writing process

During the preparation of this work the authors used ChatGPT in order to improve the readability and language of the manuscript. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the published article.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This work was supported in part by the University of Moratuwa, Sri Lanka, under the Senate Research Committee Grant SRC/LT/2026/02.

References

[1] R. Krishnan, C. Dhara, T. Horinouchi, C.K. Gotangco Gonzales, A. Dimri, M.S. Shrestha, P. Swapna, M. Roxy, S.-W. Son, D. Ayantika, et al., Compound weather and climate extremes in the Asian region: science-informed recommendations for policy, Front. Climate 6 (2025) 1504475.

[2] S. Aththanayake, C. Mallikarachchi, S. Kugarajah, J. Wickramasinghe, ResQ-Connect: An AI-powered multi-agentic platform for human-centered disaster response, 2025, https://sites.google.com/cse.mrt.ac.lk/resqconnect/. (Accessed 13 April 2026).

[3] floodsupport.org, Emergency SOS-flood rescue Sri Lanka, 2025, https://floodsupport.org/. (Retrieved 13 April 2026).

[4] C.-Y. Chang, Z. Jiang, V. Rakesh, M. Pan, C.-C.M. Yeh, G. Wang, M. Hu, Z. Xu, Y. Zheng, M. Das, et al., MAIN-RAG: Multi-agent filtering retrieval-augmented generation, in: 63rd Annual Meeting of the Association for Computational Linguistics, Vienna, Austria, 2025, pp. 2607–2622.

[5] S. Aththanayake, C. Mallikarachchi, J. Wickramasinghe, S. Kugarajah, D. Meedeniya, B. Pradhan, ResQConnect: An AI-powered multi-agentic platform for human-centered and resilient disaster response, Sustainability 18 (2) (2026) 1–48, 1014.

[6] Qwen Team, Qwen2.5: A party of foundation models, 2024, https://qwenlm.github.io/blog/qwen2.5/. (Retrieved 13 April 2026).

[7] S. Aththanayake, C. Mallikarachchi, S. Kugarajah, J. Wickramasinghe, Synthetic citizen help requests dataset for natural disasters, 2025, https://docs.google.com/spreadsheets/d/1xW_PqC9sx7Zyyd1zpq_u0mKpywYoesbfMmRJ4ufqajo/. (Accessed: April 2026).